to add up as the test is read and executed. The cost really increases if the test fails when there is no underlying problem with the code.

The value can come when the test is written if it is part of a process to create the correct code more quickly. The value can also come as a replacement for manual testing, and the value can be added later on when the test fails in a case where it indicates a real problem with new code. Longer term, tests can add value if they encourage a design style that makes code change easier.

Let's look at some example tests. The Shoulda gem provides a series of matchers of the form `<Class> should belong_to :users`.[1] That's the whole test. This test is low cost. It takes very little time to write, it's easy to understand, and it executes quickly. But it's also low value—it's tied to the database, not the code logic, so it doesn't say much about the design of the code. It's also very unlikely to fail in isolation. If the association with `users` goes away, many tests are likely to fail, so the failure of this test probably isn't telling you anything that you don't already know about the state of the code.

Conversely, imagine an end-to-end system test written with Capybara, similar to the ones you wrote in the last chapter. This test is high cost. It requires a lot of data, a number of testing steps, and complex output matching. At the same time, the test could be high value. It might be your only way to find out if all the different pieces collaborating in the checkout are communicating correctly, which is to say a test like this might find errors that no other test in the system will flag. It might also be faster to run the test by itself when developing than to walk through the checkout manually.

So the goal in keeping your test suite happy over the long haul is to minimize the costs of tests and maximize the value.

Here are some concrete steps you can take:

- Instead of thinking about what will make a test pass, think about what will make it fail. If there's no way to make the test fail that won't make an existing test fail, maybe you don't need the test.
- Think of integration tests that will help you automate a series of actions that are useful while developing the code, so you get time savings from running the integration test.
- For unit tests, write the test so that it invokes the minimal amount of code needed to make the test fail. For example, as you'll see in Chapter 7, *Using Test Doubles as Mocks and Stubs*, on page 129, error cases often

1. https://github.com/thoughtbot/shoulda